

1 Parâmetros de Configuração dos Periféricos

Com base no desenvolvimento do firmware, os seguintes parâmetros foram configurados:

1.1 Configurações do ADC (Conversor Analógico-Digital)

- **Resolução:** 12 bits (ADC_BITWIDTH_12), permitindo uma escala de 0 a 4095.
- **Atenuação:** 12 dB (ADC_ATTEN_DB_12). Esta configuração é essencial para que o ADC suporte a leitura de tensões na faixa completa de até 3.3V.
- **Canal:** ADC Unidade 1, Canal 6 (GPIO34).

1.2 Configurações do LEDC (Controle de PWM)

- **Resolução do Duty Cycle:** 13 bits (LEDC_TIMER_13_BIT), resultando em uma escala de 0 a 8191.
- **Frequência:** 5000 Hz (5 kHz), garantindo uma modulação suave para o brilho do LED.

1.3 Equação de Conversão (Valor Bruto para Tensão)

A conversão do valor digital bruto para tensão em milivolts (mV) segue a equação abaixo:

$$V_{out}(mV) = \frac{ADC_{raw} \times 3300}{4095} \quad (1)$$

Onde:

- V_{out} é a tensão calculada em mV.
- ADC_{raw} é o valor lido pelo periférico (0 a 4095).
- 3300 representa a tensão de referência de 3.3V.

2 Firmware Desenvolvido

O firmware abaixo foi desenvolvido utilizando o ESP-IDF e contempla todos os requisitos de leitura, controle de brilho e função "Hold" via botão.

```
1
2
3 #include <stdio.h>
4 #include "freertos/FreeRTOS.h"
5 #include "freertos/task.h"
6 #include "esp_adc/adc_oneshot.h"
7 #include "driver/ledc.h"
8 #include "driver/gpio.h"
9 #include "esp_log.h"
10
11 /* pinos */
12 #define POT_ADC_CHANNEL ADC_CHANNEL_6 /* GPIO34 */
13 #define POT_ADC_UNIT ADC_UNIT_1
```

```

14 #define LED_GPIO          10
15 #define BUTTON_GPIO       15
16
17 /* ADC */
18 #define ADC_MAX_VALUE      4095
19
20 /* LEDC PWM */
21 #define LEDC_TIMER         LEDC_TIMER_0
22 #define LEDC_MODE          LEDC_LOW_SPEED_MODE
23 #define LEDC_CHANNEL       LEDC_CHANNEL_0
24 #define LEDC_DUTY_RES      LEDC_TIMER_13_BIT
25 #define LEDC_FREQUENCY     5000
26 #define LEDC_DUTY_MAX      8191
27
28 #define PRINT_INTERVAL_MS  500
29
30 static const char *TAG = "ADC_PWM";
31
32
33 static void ledc_init(void)
34 {
35     ledc_timer_config_t timer_cfg = {
36         .speed_mode      = LEDC_MODE,
37         .timer_num       = LEDC_TIMER,
38         .duty_resolution = LEDC_DUTY_RES,
39         .freq_hz         = LEDC_FREQUENCY,
40         .clk_cfg         = LEDC_AUTO_CLK,
41     };
42     ESP_ERROR_CHECK(ledc_timer_config(&timer_cfg));
43
44     ledc_channel_config_t channel_cfg = {
45         .speed_mode = LEDC_MODE,
46         .channel     = LEDC_CHANNEL,
47         .timer_sel   = LEDC_TIMER,
48         .intr_type   = LEDC_INTR_DISABLE,
49         .gpio_num    = LED_GPIO,
50         .duty        = 0,
51         .hpoint      = 0,
52     };
53     ESP_ERROR_CHECK(ledc_channel_config(&channel_cfg));
54 }
55
56
57 static void button_init(void)
58 {
59     gpio_config_t io_cfg = {
60         .pin_bit_mask = (1ULL << BUTTON_GPIO),
61         .mode          = GPIO_MODE_INPUT,
62         .pull_up_en    = GPIO_PULLUP_ENABLE,
63         .pull_down_en  = GPIO_PULLDOWN_DISABLE,
64         .intr_type     = GPIO_INTR_DISABLE,
65     };
66     ESP_ERROR_CHECK(gpio_config(&io_cfg));
67 }
68
69
70 static uint32_t adc_to_duty(int raw)
71 {

```

```

72     return (uint32_t)((raw * (uint32_t)LEDC_DUTY_MAX) / ADC_MAX_VALUE);
73 }
74
75
76 static uint32_t adc_to_mv(int raw)
77 {
78     return (uint32_t)((raw * 3300UL) / ADC_MAX_VALUE);
79 }
80
81
82 void app_main(void)
83 {
84     adc_oneshot_unit_handle_t adc_handle;
85
86     adc_oneshot_unit_init_cfg_t unit_cfg = {
87         .unit_id = POT_ADC_UNIT,
88         .ulp_mode = ADC_ULP_MODE_DISABLE,
89     };
90     ESP_ERROR_CHECK(adc_oneshot_new_unit(&unit_cfg, &adc_handle));
91
92     adc_oneshot_chan_cfg_t chan_cfg = {
93         .atten = ADC_ATTEN_DB_12,
94         .bitwidth = ADC_BITWIDTH_12,
95     };
96     ESP_ERROR_CHECK(adc_oneshot_config_channel(adc_handle,
97         POT_ADC_CHANNEL, &chan_cfg));
98
99     ledc_init();
100     button_init();
101
102     bool hold_mode = false;
103     bool btn_last_state = true;
104     int frozen_raw = 0;
105
106     TickType_t last_print = xTaskGetTickCount();
107
108     ESP_LOGI(TAG, "Sistema iniciado. GPIO34=POT | GPIO%d=LED | GPIO%d=
109         BTN",
110         LED_GPIO, BUTTON_GPIO);
111
112     while (1) {
113         /* Detec o de borda do bot o com debounce */
114         bool btn_now = gpio_get_level(BUTTON_GPIO);
115
116         if (btn_last_state == true && btn_now == false) {
117             vTaskDelay(pdMS_TO_TICKS(20));
118             if (gpio_get_level(BUTTON_GPIO) == false) {
119                 hold_mode = !hold_mode;
120                 if (hold_mode) {
121                     adc_oneshot_read(adc_handle, POT_ADC_CHANNEL, &
122                         frozen_raw);
123                     ESP_LOGI(TAG, ">>> MODO HOLD ativado (raw=%d)",
124                         frozen_raw);
125                 } else {
126                     ESP_LOGI(TAG, ">>> MODO LIVE ativado");
127                 }
128             }
129         }
130     }
131 }

```

```

126     btn_last_state = btn_now;
127
128     /* Leitura ADC */
129     int raw;
130     if (hold_mode) {
131         raw = frozen_raw;
132     } else {
133         ESP_ERROR_CHECK(adc_oneshot_read(adc_handle, POT_ADC_CHANNEL
134                                     , &raw));
135
136     /* duty */
137     uint32_t duty = adc_to_duty(raw);
138     ledc_set_duty(LED_MODE, LEDC_CHANNEL, duty);
139     ledc_update_duty(LED_MODE, LEDC_CHANNEL);
140
141     /* print */
142     TickType_t now = xTaskGetTickCount();
143     if ((now - last_print) >= pdMS_TO_TICKS(PRINT_INTERVAL_MS)) {
144         last_print = now;
145         uint32_t voltage_mv = adc_to_mv(raw);
146         printf("[%s] raw=%4d tensao=%4lu mV duty=%4lu\n",
147             hold_mode ? "HOLD" : "LIVE",
148             raw,
149             (unsigned long)voltage_mv,
150             (unsigned long)duty);
151     }
152
153     vTaskDelay(pdMS_TO_TICKS(10));
154 }
155 }

```

Listing 1: Implementação ADC e PWM no ESP32

3 Monitoramento

O sistema imprime no terminal a cada 500ms o valor bruto do ADC, a tensão calculada e o estado atual (LIVE ou HOLD).